

LLMConf: Knowledge-Enhanced Configuration Optimization for Large Language Model Inference

Jingkai He^{1†} Pengfei Chen^{2,3*} Yilun Wang^{1†} Haiyu Huang^{2†} Chuanfu Zhang^{1*} Haojia Huang^{2†} Danwen Chen^{2†}

¹ School of Systems Science and Engineering, Sun Yat-sen University, Guangzhou, China

² School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou, China

³ Guangdong Key Laboratory of Big Data Analysis and Processing, Guangzhou, 510006, China

[†] {hejk25, wangylun6, huanghy95, huanghj78, chendw9}@mail2.sysu.edu.cn

^{*} {chenpf7, zhangchf9}@mail.sysu.edu.cn

Abstract—As large language models (LLMs) are widely applied across various domains, improving the quality of LLM inference services is essential. In this paper, we find that optimizing configuration parameters of LLM inference engines can significantly improve LLM inference performance in terms of latency and throughput. Therefore, we propose LLMConf, an automated performance tuning system that optimizes multiple LLM inference performance metrics by searching for the optimal configuration parameters of the LLM inference engine. We first introduce a knowledge-enhanced approach to identify the set of configuration parameters (LLMConfigs) that most significantly impact LLM performance from the adjustable parameters provided by the LLM inference engine. We then perform automated data collection to build functional relationships between LLMConfigs and each performance metric. Additionally, LLMConf employs a multi-objective optimization module to obtain optimal LLMConfigs for simultaneously optimizing multiple performance metrics. The experimental results show that LLMConf significantly outperforms existing methods. Compared to the default configuration parameters of the LLM inference engine, LLMConf achieves an average improvement of 20.1% across 7 key performance metrics. Moreover, experiments demonstrate that LLMConf has strong transferability across diverse datasets, varying concurrency levels and different LLM base models.

Index Terms—Large Language Model, Inference Engine, Configuration Optimization, Knowledge Augmentation

I. INTRODUCTION

Large Language Models (LLMs) have emerged as transformative tools across various fields [1], demonstrating excellent performance and versatility. However, due to the large number of parameters in LLMs, they require significant computational and storage resources [2], making local deployment challenging. Cloud providers have begun providing services for deploying and inferring LLMs. The LLM inference services provided by cloud service providers run LLM instances using advanced inference engines such as vLLM [3]. LLM inference engines like vLLM offer adjustable parameters, and tuning these can flexibly schedule integrated technologies, affecting inference performance.

The performance of LLM inference is crucial for both users and cloud service providers [4]. Existing LLM deployment solutions typically use default parameters of the LLM inference engine or empirically set the parameters [5]. Through research,

we find that tuning parameters of the LLM inference engine can significantly improve LLM inference performance.

Although parameter tuning methods have achieved good results in many fields [6], tuning the configuration parameters of LLM inference engines has its own uniqueness. First, the optimization goals are different, as LLM inference focuses more on optimizing latency and throughput in terms of tokens rather than requests. Second, LLM inference involves many dynamic and nonlinear factors with complex interactions, making it more challenging than other areas. Moreover, due to the complexity of LLM inference engine, the configuration parameters involve various aspects such as memory allocation strategies and request scheduling. As a result, the number of parameters that need to be tuned is greater than in other scenarios. The primary challenges of tuning LLM inference engine parameters are shown as follows.

C1: Challenge in capturing parameter effects on LLM inference performance. When changing the configuration parameters of the LLM inference engine, the specific changes in LLM inference performance are difficult to predict in advance and require experiments to determine. This causes cold start issues, leading to resource waste and poor model inference performance.

C2: Challenge in identifying the performance-critical tuning parameters and their scope. Due to the lack of sufficient practical experience and detailed guidance manuals for LLM configuration parameters, it is challenging for us to determine specific tuning parameters and the value ranges of various configuration parameters. Moreover, as the technology of LLM inference engines advances, the value ranges of parameters frequently change. However, not reducing the number and range of tuning parameters will expand the search space, lowering optimization performance [7].

C3: Challenge in optimizing multiple performance metrics simultaneously. A single performance metric often fails to reflect the performance of LLMs under different scenarios. Therefore, we select multiple performance metrics for optimization, which actually constitutes a multi-objective optimization problem. However, when addressing high-dimensional optimization problems, existing optimization algorithms [8] often tend to get trapped in local optima and

exhibit slow convergence rates.

LLMConf Approach. To address the gaps in existing research, we propose LLMConf, an optimization method for **LLM** inference that focuses on tuning **Configuration** parameters. This method aims to recommend an optimal set of configuration parameters to improve LLM inference performance. LLMConf utilizes a constructed knowledge base through In-Context Learning (ICL) [9] to guide GPT-4 in identifying configuration parameters and providing the corresponding data types and value ranges. The knowledge-enhanced approach effectively supplements the lack of information regarding LLM configuration parameters. As the technology of LLM inference engines evolves, the knowledge base can be updated at any time, effectively addressing the issues of parameter selection and value range determination (**solution to C2**). To achieve accurate predictions of performance metrics under different configuration parameter values, we use the TPOT [10] model, which is an automatic machine learning tool to construct the optimal ML model, to establish the functional relationships between configuration parameters and each performance metric based on the collected data. This allows us to understand how changes in parameter values affect LLM inference performance (**solution to C1**). Additionally, we implement a multi-objective optimization module based on NSGA-III, which can simultaneously optimize multiple performance metrics, thereby achieving a comprehensive enhancement of LLM inference performance (**solution to C3**).

In summary, our contributions are as follows.

- We use a knowledge-enhanced approach to identify the key configuration parameters and their value ranges, both of which are crucial for improving LLM inference performance.
- We propose a multi-objective tuning method for configuring parameters of LLM inference engines called LLMConf, which optimizes multiple LLM performance metrics based on an improved NSGA-III algorithm [11, 12], enhancing the efficiency of searching for optimal solutions. The source code of LLMConf is available at <https://github.com/IntelligentDDS/LLMConf>.
- The experimental results confirm that LLMConf outperforms state-of-the-art methods [13] for tuning the configuration parameters of LLM inference engines, achieving an average 20.1% improvement across 7 metrics.

II. BACKGROUND AND MOTIVATION

A. Background

LLM Inference Engine and Parameters. The development of LLM inference engines has been progressing rapidly, with advanced inference engines such as vLLM [3] and TensorRT-LLM [14]. We choose vLLM as the inference engine because it is widely used [15] and a fast, easy-to-use library for LLM inference and serving. The LLM inference engine integrates various technologies and exposes numerous adjustable parameters. For vLLM, these parameters include integer variables like *max-num-batched-tokens*, which limits

the number of tokens processed per batch through continuous batching technology; enumeration variables like *block-size*, which controls the context length allocated to the model via memory block allocation; float variables like *gpu-memory-utilization*, which improves GPU utilization through GPU memory management [16]; and boolean variables like *enable-chunked-prefill*, which divides the sequence into chunks using chunked prefill technology.

Metrics Optimization of LLM inference. We evaluate the inference performance of LLMs in terms of latency and throughput. Latency-related metrics include *latency_average*, which represents the mean response time taken to complete each request, and *latency_p99*, which captures the 99th percentile latency, reflecting the worst-case tail delays. Additionally, *time_to_first_token_average* (*TTFT_average*) and *time_to_first_token_p99* (*TTFT_p99*) measure the initial response delay, with the former indicating the average time and the latter reflecting the 99th percentile. *time_per_output_token_average* (*TPOT_average*) and *time_per_output_token_p99* (*TPOT_p99*) assess per-token generation efficiency, highlighting average and 99th percentile times. Throughput is evaluated through *tokens_per_second_average* (*TPS_average*), which tracks tokens processed per second. We aim to improve the quality of LLM inference services by optimizing these metrics. Genetic algorithms are widely used in multi-objective optimization tasks [17]. Genetic algorithms perform parallel search in the global solution space through population evolution, using non-dominated sorting and diversity preservation strategies to effectively approximate the Pareto optimal set.

B. Motivations

In this section, we present our experimental observations and introduce our motivations.

Motivation 1: The configuration parameters of the LLM inference engine have a significant impact on LLM inference performance, and the optimal configuration parameters vary across different scenarios. Fig.1(a) presents the experimental results based on the Llama3-8B. We set the concurrency levels to three gradients: 20, 50 and 80. The dataset chosen for the experiment is the *GSM8K*¹ released by OpenAI. Conf.A, Conf.B and Conf.C represent three different sets of configuration parameters respectively.

We observe that in scenarios with concurrency levels of 20, 50 and 80, the optimal configuration can increase throughput by 8.3%, 25.8% and 15.8%, respectively. This demonstrates that tuning configuration parameters can enhance the inference performance of LLM, and therefore, we aim to improve LLM inference service quality through performance tuning. Additionally, we observe that the optimal configuration parameters in one scenario may even lead to degraded inference performance in another scenario. Thus, no single configuration works best in all scenarios, and it is essential to perform performance tuning tailored to each scenario.

¹<https://huggingface.co/datasets/openai/gsm8k>

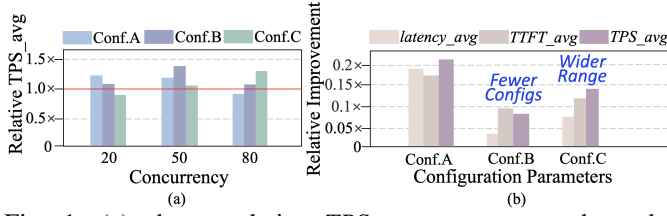


Fig. 1: (a) shows relative *TPS_average* compared to the default value of vLLM (higher is better). (b) shows the performance improvements in *latency_average*, *TTFT_average* and *TPS_average* compared to the default vLLM parameter configuration (higher is better).

Motivation 2: Insufficient selection of configuration parameters and determination of their ranges will lead to suboptimal performance in the tuning process. The experimental results in Fig.1(b) are based on the Llama3-8B with a uniform concurrency of 50 and the *GSM8K* dataset. Conf.B has fewer tuned parameters but the same value ranges as Conf.A. Conf.C maintains the same number of tuned parameters as Conf.A but with a wider range.

Compared to Conf.A, Conf.B achieves performance decreases of 15.3%, 7.6% and 13.6% in *latency_average*, *TTFT_average* and *TPS_average*, respectively, while Conf.C achieves decreases of 12.3%, 4.1% and 8.8%. The reasons for the above experimental results can be attributed to: (i) an insufficient selection of configuration parameters, leaving some important configurations untuned, (ii) the wider range of parameter values expands the search space, making it more difficult to find the optimal solution and (iii) the wider value range fails to consider inter-parameter constraints (e.g., the maximum number of sequences in a batch (*max-num-seqs*) is set lower than the maximum number of tokens in a batch (*max-num-batched-tokens*) to prevent the product of sequences and tokens per sequence from exceeding the allowed total token count). When parameter tuning induces constraint-triggered errors, the vLLM engine aborts the inference process and raises a value domain exception. These are also shortcomings of the current state-of-the-art inference engine tuning system [13]. Therefore, we aim to leverage the LLM in conjunction with relevant external knowledge to determine which parameters need tuning and their respective value ranges.

Motivation 3: A set of optimal configuration parameters should be determined by multiple objectives. The results in Fig.2 are from the Llama3-8B with a concurrency of 50, using the *GSM8K* dataset. Three tuning trials on the same configuration set produce Conf.A, Conf.B and Conf.C.

We observe that Conf.A, Conf.B and Conf.C achieve average performance improvements of 21.4%, 20.3% and 16.7% respectively across three performance metrics. Customers' requirements for LLM inference services vary across different application scenarios. For instance, customers using real-time simultaneous interpretation systems demand lower latency, while those utilizing offline recommendation systems prioritize higher request throughput. Therefore, high-quality LLM inference services should not be limited to optimizing a single performance metric [18]. Given this, we consider

multiple optimization objectives when tuning the configuration parameters to achieve a more comprehensive improvement in the quality of LLM inference services.

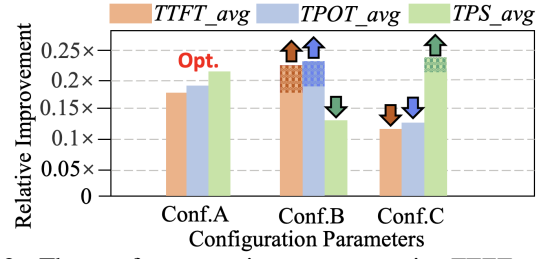


Fig. 2: The performance improvements in *TTFT_average*, *TPOT_average* and *TPS_average* compared to the default vLLM parameter configuration (higher is better).

III. LLMCONF OVERVIEW

LLM Inference Performance Optimization Configuration Tuning. vLLM exposes many parameters, including $C^1, C^2, \dots, C^n$. We define the configuration space $\Omega = \Omega_1 \times \Omega_2 \times \dots \times \Omega_k$, where Ω_i represents the value range of the i th tuning parameter. Vector $\mathbf{C}$ represents the configuration parameters that we tune, where $\forall C[i] \in \{C^1, C^2, \dots, C^n\}$. Additionally, the i th element $C[i] \in \Omega_i$, representing the value of the i th parameter. We use $L_i(\mathbf{C})$ to represent the objective function of the i th latency-related metric for $\mathbf{C}$ and $T_j(\mathbf{C})$ to represent the objective function of the j th throughput-related metric for $\mathbf{C}$. We define a nonlinear function $\mathcal{Y}(\mathbf{C}) = \sum_{i=1}^N L_i(\mathbf{C}) + \left(-\sum_{j=1}^M T_j(\mathbf{C})\right)$. Furthermore, we use $\lambda = \{\lambda_1, \lambda_2, \dots, \lambda_m\}$ to represent the set of known constraints, where λ_i denotes the i th known constraint that restricts the relationship between the elements of $\mathbf{C}$. We aim to find the optimal configuration parameters that simultaneously optimize both latency and throughput metrics:

$$\mathbf{C}^* = \arg \min_{\mathbf{C} \in \Omega} \mathcal{Y}(\mathbf{C}) \quad (1)$$

$$\text{s.t. } \lambda_i, \forall \lambda_i \in \lambda \quad (2)$$

Overview. We depict the overall architecture of LLMConf in Fig.3. The core of LLMConf is to efficiently leverage the inference techniques integrated into the inference engines by tuning the selected configuration parameters, thereby impacting the inference performance of LLMs.

Initially, we build a knowledge base for LLMConf. We then use In-context Learning [9] to incorporate knowledge from the knowledge base into prompts, guiding GPT-4 to infer the parameters that need tuning, along with their value ranges and data types. We use **LLMConfigs** to represent the parameters that need tuning. We then collect data offline by deploying a combination of configuration parameters into the LLM Platform and starting the vLLM service each time. We test the inference performance of the LLM using a series of performance metrics. Each time, the performance metrics and corresponding configuration parameter values need to be

recorded②. Subsequently, we perform performance modeling based on the collected data③. We utilize the TPOT model to construct the objective function of each performance metric for LLMConfigs based on the collected data④. The multi-objective optimization module implemented based on the NSGA-III algorithm can simultaneously optimize multiple performance metrics⑤. Finally, we derive the optimal configuration parameters, which enhance the inference performance of the LLM comprehensively.

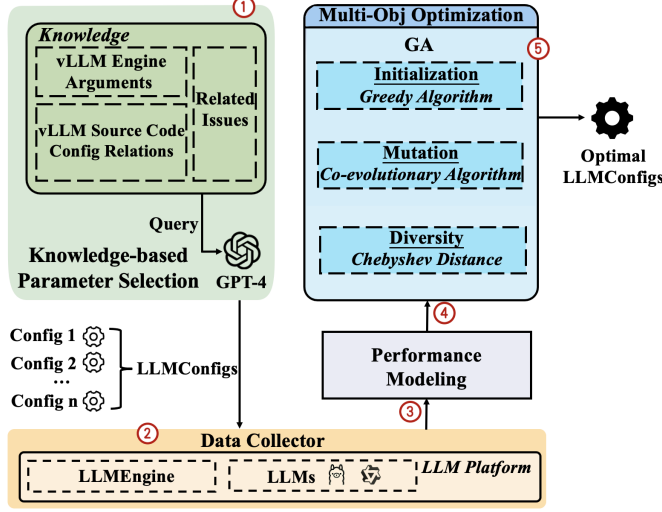


Fig. 3: The architecture of LLMConf.
IV. DESIGN OF LLMCONF

A. Knowledge-based Parameter Selection

Fig.1(b) shows that although the configuration parameters in the LLM inference engine all have default values, they do not achieve optimal inference performance, which demonstrates the significance of tuning parameters. Fig.1(b) also shows that determining the tuning targets and their value ranges is equally important. In order to avoid overlooking potentially effective configurations and to enhance the search efficiency for the optimal solution, we determine the tuning parameters and their ranges using the LLM, because its comprehensive knowledge enables it to output reliable responses while saving time.

To improve the accuracy of LLM outputs, we build a knowledge base as supplementary information. Our knowledge base comprises the following three parts:

vLLM Engine Arguments. The Engine Arguments documentation is obtained from the vLLM [3] open-source project and it includes meanings and default values for a total of 83 configuration parameters.

Relationships Among Configuration Parameters in vLLM Source Code. We summarize 59 relationships among configuration parameters from vLLM's open-source code [3].

Related Issues. We crawl all 13700 issues from the vLLM [3] repository on GitHub and manually verify each one. If an issue's content is related to the configuration parameters in Engine Arguments, we add its main content and remove information already in the knowledge base. There are a total of 69 related issues in the knowledge base.

We input the entire knowledge base as the prompt (about 9968 tokens) into GPT-4 to determine the tuning parameters. For determining the value range of a specific configuration parameter, we input both the LLM platform configuration information and the related information from the knowledge base (average 880 tokens) into GPT-4. We also incorporate self-consistency to enhance the accuracy of the inference. For the detailed content of the prompt, please refer to the source code. We uniformly set the *temperature* to its default value and the length of all prompts is kept below GPT-4's maximum allowable context length. The results output by GPT-4 are shown in Table I, where M denotes *max-position-embeddings* (the maximum sequence length that the model can handle).

TABLE I: PARAMETERS TO TUNE.

Configuration Parameter	Type	Range
tensor-parallel	Integer	[1, #GPUs]
max-num-batched-tokens	Integer	[4000, #M]
max-num-seqs	Integer	[64, 2048]
swap-space	Integer	[1, 8]
block-size	Enumeration	{8, 16, 32}
scheduler-delay-factor	Float	[0, 2]
gpu-memory-utilization	Float	[0.7, 1]
enable-chunked-prefill	Boolean	{True, False}
enable-prefix-caching	Boolean	{True, False}
disable-custom-all-reduce	Boolean	{True, False}
use-v2-block-manager	Boolean	{True, False}

Our experiments in Sec.V-B3 show that the inference results of GPT-4 are similar to the conclusions derived from the data-driven Random Forest, which validates the correctness of our approach. We also save the time typically required for the tedious parameter selection process in traditional work [19].

B. Data Collector

Frequent redeployment of LLM to assess inference service quality across different combinations of configuration parameter values incurs huge resource wastage. Therefore, we develop a predictive modeling framework for estimating LLM inference service quality evaluation metrics without physical deployment. However, accurate modeling requires substantial volumes of training data. Below, we introduce the data collection process.

The data collection process maintains fixed concurrency levels and benchmark datasets for the deployed LLM, iteratively collecting 1500 data samples:

$$\mathcal{D} = \bigcup_{k=1}^{1500} \left\{ \underbrace{\mathbf{c}_k}_{\substack{\text{Sampling} \\ \mathbf{c}_k \in \Lambda}} \mapsto \underbrace{\Phi(\mathbf{c}_k)}_{\text{Benchmarking}} \mapsto \underbrace{(\mathbf{c}_k, \Phi(\mathbf{c}_k))}_{\text{Logging}} \right\}, \quad (3)$$

where the parameter space Λ is defined as $\Lambda = \Lambda_1 \times \Lambda_2 \times \dots \times \Lambda_{11}$, $\mathbf{c}_k = [c_{k,1}, \dots, c_{k,11}]^T$, and $\Phi(\mathbf{c}_k) = [\phi_1(\mathbf{c}_k), \phi_2(\mathbf{c}_k), \dots, \phi_7(\mathbf{c}_k)]^T$.

In the configuration parameter vector $\mathbf{c}_k$ of the k -th experiment, each configuration parameter $c_{k,j}$ is randomly assigned a value within its respective value range Λ_j with the specific values of Λ_j shown in Table I.

After finalizing configuration parameters, the benchmarking stage evaluates LLM inference service quality in terms of latency and throughput. Each metric is computed via Eq. 4 using N requests, where test inputs $\mathbf{p}_i$ and configuration-dependent responses $\mathbf{r}_i(\mathbf{c}_k)$ are processed by g_j to calculate per-request values:

$$\phi_j(\mathbf{c}_k) = \frac{1}{N} \sum_{i=1}^N g_j(\mathbf{c}_k, \mathbf{p}_i, \mathbf{r}_i(\mathbf{c}_k)). \quad (4)$$

A total of 1500 data records are collected through the above process, with each linking configuration parameters to the corresponding performance metrics, represented as $(\mathbf{c}_k, \Phi(\mathbf{c}_k))$.

C. Performance Modeling

The modeling process aims to learn the function F_j that maps the configuration parameters $\mathbf{c}_k$ to the corresponding performance metric $\phi_j(\mathbf{c}_k)$. We construct such a F_j for each performance metric:

$$\phi_j(\mathbf{c}_k) = F_j(\mathbf{c}_k; \theta_j), \quad (5)$$

where θ_j represents the model parameters learned for each metric.

Next, we present an automated regression framework using TPOT [10] model to construct the predictive function for each performance metric. The independent variables consist of 11 configuration parameters, while the dependent variables include 7 performance metrics. For each metric y_i , TPOT searches the optimal pipeline f_i through genetic programming:

$$f_i^* = \operatorname{argmin}_{f \in \mathcal{F}} \left(\frac{1}{N} \sum_{j=1}^N \left(y_i^{(j)} - f(\mathbf{X}^{(j)}) \right)^2 \right), \quad (6)$$

where $\mathcal{F}$ denotes the space of scikit-learn pipelines, N represents total training samples, and $\mathbf{X} \in \Lambda$ represents the normalized configuration parameters.

We standardize configuration parameters and performance metrics using z-score normalization $z = \frac{x-\mu}{\sigma}$, where μ and σ denote the mean and standard deviation of each variable, and split data for unbiased evaluation. TPOT evolves populations to optimize Mean Squared Error (MSE), with top-performing pipelines persisted alongside inverse scalars. Each prediction function maps raw configuration inputs to destandardized metric values through a saved pipeline and scalars. Model quality is quantified via R^2 and RMSE, ensuring reliable empirical performance estimation.

Accurate modeling enables precise prediction of performance metrics across various configuration parameters, helping to effectively capture the impact of different configuration parameter combinations on LLM inference performance.

D. Multi-Objective Optimization

Given the diverse customer applications, we optimize seven performance metrics across latency and throughput to ensure

high-quality LLM inference service. We first construct predictive functions for each performance metric before optimization, as this effectively filters out configuration combinations that could lead to system crashes, ensuring system stability [20]. Additionally, it prevents costly direct system evaluations during optimization iterations, thus reducing computational costs [21].

We propose a more effective optimization algorithm based on the NSGA-III [11, 12] algorithm for multiple objectives. After several iterations, the optimal configuration parameters $\mathbf{x}^*$ obtained minimize the predictive function values for latency-related metrics and maximize those for throughput-related metrics:

$$\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x} \in \Lambda} [\vartheta_1(\mathbf{x}) + \dots + \vartheta_6(\mathbf{x}) - \psi(\mathbf{x})], \quad (7)$$

where the set of latency-related objective functions Θ is defined as $\Theta = \{\vartheta_i\}_{i=1}^6$ and the set of throughput-related objective functions Ψ is defined as $\Psi = \{\psi\}$.

Non-dominated Sorting Genetic Algorithm III (NSGA-III) is a multi-objective optimization algorithm. It filters out a set of high-quality candidate solutions through non-dominated sorting and introduces reference points to ensure that the solutions uniformly cover the entire Pareto front. The algorithm then combines selection, crossover and mutation operations from genetic algorithms to optimize multiple objectives. To improve the performance of the NSGA-III algorithm in optimizing high-dimensional objectives, we make the following three improvements.

Algorithm 1 NSGA-III-based Multi-objective Optimization

```

1: Input:  $n_{obj}, n_{var}, lb, ub, npop, ngen, ref\_points$ 
2: Output: Final population  $pop$ , final objective values  $objs$ 
   // Initialize population using a greedy algorithm
3:  $pop \leftarrow \text{greedy\_initialization}(npop, nvar, lb, ub)$ 
4:  $objs \leftarrow \text{cal\_obj}(pop, n_{obj})$ 
5:  $z_{min} \leftarrow \min(objs)$ 
6: for  $gen = 1$  to  $ngen$  do
7:    $pfs, rank \leftarrow \text{nd\_sort}(objs)$ 
8:    $mating\_pool \leftarrow \text{selection}(pop, pc, rank)$ 
9:    $offsprings \leftarrow \text{crossover}(mating\_pool, lb, ub, pc, \eta_c)$ 
   // Coevolutionary mutation using information from other individuals
10:   $offsprings \leftarrow \text{coevolutionary\_mutation}(offsprings,$ 
11:     $lb, ub, pm)$ 
12:   $new\_objs \leftarrow \text{cal\_obj}(offsprings, n_{obj})$ 
13:   $combined\_pop \leftarrow \text{concatenate}(pop, offsprings)$ 
14:   $combined\_objs \leftarrow \text{concatenate}(objs, new\_objs)$ 
   // Environmental selection using Chebyshev distance
15:   $pop, objs \leftarrow \text{environmental\_selection}(combined\_pop,$ 
16:     $combined\_objs, z_{min}, npop, ref\_points)$ 
17:   $z_{min} \leftarrow \min(objs)$ 
18: end for
19: return  $pop, objs$ 

```

Greedy algorithm for population initialization. Random initialization in NSGA-III can cause uneven population distribution, hindering exploration of high-quality solutions with multiple objectives. We introduce a greedy algorithm for population initialization, selecting better solutions to form the initial population. The greedy initialization (Line 3 in Algorithm 1) generates a candidate pool via random sampling within the range of configuration parameters, then selects the top- n_{pop} configuration parameters minimizing the aggregated prediction objectives, thus forming the initial population $\mathcal{P}_0$:

$$\mathcal{P}_0 = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_k\}, \quad (8)$$

where $\Gamma(\mathbf{x}) = \sum_{\vartheta \in \Theta} \vartheta(\mathbf{x}) - \sum_{\psi \in \Psi} \psi(\mathbf{x})$, $\Gamma(\mathbf{x}_1) \leq \Gamma(\mathbf{x}_2) \leq \dots \leq \Gamma(\mathbf{x}_k) \leq \dots \leq \Gamma(\mathbf{x}_{|\mathcal{S}_{\text{cand}}|})$, $\mathbf{x}_i$ denotes the i th best configuration in the candidate set $\mathcal{S}_{\text{cand}}$, and $|\mathcal{P}_0| = n_{\text{pop}}$.

Coevolutionary algorithm for mutation. NSGA-III uses polynomial mutation to enhance population exploration, but as objectives grow, the search space expands exponentially, limiting exploration of uncommon regions. We introduce a coevolutionary algorithm to enhance interaction between populations, using the characteristic information of other individuals to mutate the current individual. The co-evolutionary mutation (Lines 10-14 in Algorithm 1) differs for continuous and discrete parameters. For continuous ones, perturb the values of randomly selected individuals by $\pm 10\%$ of their allowed range and truncate to the valid interval, while for discrete ones, values are redrawn from their value sets:

$$\mathbf{x}'_i[k] = \begin{cases} \mathbf{x}_j[k] + \delta_k (ub_k - lb_k) & \text{if } k \in \mathcal{C} \\ d_k \in \Lambda_k & \text{if } k \in \mathcal{D} \end{cases} \quad (9)$$

$$\text{S.t. } lb_k \leq \mathbf{x}'_i[k] \leq ub_k, \quad (10)$$

where perturbation factor $\delta_k \sim \mathcal{U}(-0.1, 0.1)$, $\mathcal{C}$ and $\mathcal{D}$ represent continuous and discrete parameters.

Chebyshev distance for measuring the gap between individuals and the optimal solution. NSGA-III employs vertical distance to measure one-dimensional gaps to the optimum, potentially obscuring overall performance in high-dimensional spaces. We introduce the Chebyshev distance in environmental selection (Lines 15-17 in Algorithm 1), where the individuals with the smallest Chebyshev distance are the ones closest to the ideal solution in the current population. The Chebyshev distance $T(\mathbf{x})$ quantifies the worst-case normalized deviation of a configuration's performance from the ideal point $z_{\min}$, which is dynamically updated as the best observed value for each objective:

$$T(\mathbf{x}) = \max \left\{ \max_{1 \leq i \leq 6} \frac{\vartheta_i(\mathbf{x}) - z_{\min}^{(\vartheta_i)}}{\sigma_{\vartheta_i}}, \frac{z_{\min}^{(\psi)} - \psi(\mathbf{x})}{\sigma_{\psi}} \right\}, \quad (11)$$

where normalization factors σ_{ϑ_i} and σ_{ψ} normalize objectives to comparable scales.

V. EXPERIMENTAL EVALUATION

In this section, we validate our methods in three parts: 1)Effectiveness Validation; 2)Transferability Analysis; 3)Ablation Study

A. Experimental Settings

Implementation. We implement LLMConf using Python 3.10. GPT-4 is adopted to select configuration parameters and determine their value ranges. Pytorch and vLLM [3] are used to build the foundational experimental platform for LLMs. The establishment of functional relationships is completed using sklearn and TPOT [10] model.

Computing Resources. Our experiments are conducted on a single node, and the hardware specifications are listed in Table II.

TABLE II: Hardware Specs.

Device	Model	Memory
GPU	NVIDIA A40	48GB
CPU	Intel(R) Xeon(R) Gold 6326 CPU @ 2.90GHz	125GB

LLM Inference Engine. We use vLLM-v0.5.5 as our inference engine system, and its default configuration for the parameters to be tuned is identical to that of the latest version.

LLM. We conduct experiments using Llama3-8B [22] and Qwen2.5-14B [23] because both Llama and Qwen are representative model architectures for LLMs, and 8B and 14B are common parameter sizes [24]. The fine-tuned 8B and 14B LLMs can perform well on most tasks [25, 26].

Datasets. We choose *GSM8K* released by OpenAI and *ChatDoctor-HealthCareMagic-100k* (*HealthCareMagic*) released by Lavita AI as our datasets. *GSM8K* contains approximately 8,000 grade-school-level math problems and is mainly used to evaluate the performance of LLMs in mathematical reasoning tasks. *ChatDoctor-HealthCareMagic-100k* consists of 100,000 dialogues from the HealthCareMagic platform and is designed to assess the ability of LLMs in medical question-answering and diagnostic support tasks. We choose these two datasets for their broad representativeness in different domains, making them effective for testing LLM performance across various scenarios.

Collected Data. We collect a total of 12,000 data points for different combinations of LLMs, concurrency levels, and datasets. For each combination, we implement automated data collection on the LLM Platform, gathering 1,500 data points for Performance Modeling. The data collection for each combination takes approximately 2 days, and we conduct the data collection in parallel using six NVIDIA A40 48GB GPUs in our server cluster, which takes about 4 days.

Metrics. We evaluate the inference performance of LLMs from two aspects: latency and throughput. In terms of latency, we consider *latency_average*, *latency_p99*, *TTFT_average*, *TTFT_p99*, *TPOT_average* and *TPOT_p99*. For throughput, we measure *TPS_average*.

²<https://huggingface.co/datasets/lavita/ChatDoctor-HealthCareMagic-100k>

Baselines. We compare LLMConf with the following baselines.

- **RS:** Random Search randomly chooses configuration parameters from the search space. We regard RS as a basic approach without any model.
- **SCOOT** [13] is a performance tuning method based on Bayesian Optimization (BO), utilizing single-objective BO (SOBO) and multi-objective BO (MOBO) to optimize the objectives. It introduces a parallel strategy to evaluate multiple configurations, and employs a random forest model to learn the feasibility of configurations.
- **MAB** (Multi-Armed Bandit) maximizes the reward value by exploring different combinations of configuration parameters, thereby optimizing various metrics.
- **DDPG** (Deep Deterministic Policy Gradient) combines policy learning and value function evaluation, utilizing deep neural networks to approximate Q-values, thereby exploring and optimizing configuration parameters in continuous action spaces.
- **NSGA-III** (Non-dominated Sorting Genetic Algorithm III) [11, 12] is a Genetic Algorithm (GA) that classifies individuals into different ranks through non-dominated sorting and guides the search direction using predefined reference points, ensuring that the generated Pareto front is as evenly distributed as possible.

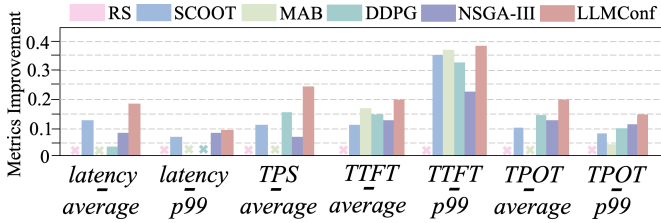


Fig. 4: Comparison of relative inference performance enhancement of Llama3-8B using different optimization algorithms (higher is better). Here, we set the concurrency level to 50 and choose the *GSM8K* dataset.

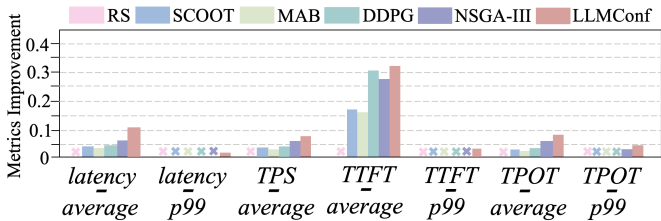


Fig. 5: Comparison of relative inference performance enhancement of Llama3-8B using different optimization algorithms (higher is better). Here, we set the concurrency level to 50 and choose the *HealthCareMagic* dataset.

B. Results and Analysis

1) **Effectiveness Validation:** We now evaluate the LLMConf's ability to search for the optimal combination of configuration parameters while ensuring the output quality of

responses. Fig.4 and Fig.5 show the enhancement of inference performance for LLMs using LLMConf across different datasets. LLMConf is compared with other multi-objective optimization algorithms on two datasets. On average, LLMConf achieves improvements of 20.1% and 8.8% across all performance metrics for Llama3-8B on *GSM8K* and *HealthCareMagic*, respectively.

However, Random Search explores all functions by randomly selecting a set of configuration parameter values within the search space, without utilizing feedback information from previous searches. SCOOT [13] is based on Bayesian optimization. In high-dimensional spaces, the complex interactions between configuration parameters can lead to certain local solutions being mistakenly considered optimal. MAB can only make decisions based on a single reward signal, making it difficult to handle multiple conflicting objectives in multi-objective optimization problems. Consequently, it struggles to find balanced solutions when objectives are in conflict. DDPG typically consists of a policy network and a value network, which are used to estimate the policy and Q-value functions, respectively. When optimizing multiple objectives simultaneously, DDPG requires different policies to handle each objective, increasing the complexity of the network. Accordingly, the gradient propagation path becomes longer, causing a decrease in convergence speed. NSGA-III [11, 12] tends to have a sparse distribution of solutions on the Pareto front in high dimensions, which makes it difficult to enhance solution diversity, resulting in insufficient exploration of the solution space. From Fig.4 and Fig.5, we can observe that some performance metrics are not optimized after using the baseline models ('x' in the figure indicates that the metric is not optimized compared to using the default configuration parameters provided by vLLM).

We also conduct experiments to verify that LLMConf optimizes LLM inference performance without compromising the quality of the generated answers. We select *BertScore* as the evaluation metric to assess the quality of the generated text. *BertScore* evaluates the similarity between two sentences by calculating the cosine similarity between their embeddings in the BERT[27] model, which takes into account both contextual and semantic information, allowing for a more accurate measurement of sentence similarity. On average, the F1 score reaches 0.875 across both datasets. This indicates that for the same question, the answers generated by the LLM using LLMConf have a higher similarity with the answers in the dataset. This is because configuration parameters mainly affect computational efficiency and resource management, while model performance and answer quality depend on model architecture, training data and the training process. Although LLMConf does not always guarantee improved answer quality, it enhances user experience by reducing inference latency and increasing throughput.

2) **Transferability Analysis:** In practical applications, concurrency levels are often influenced by factors such as user requests and business demands. Experiments demonstrate that LLMConf can effectively enhance the inference performance

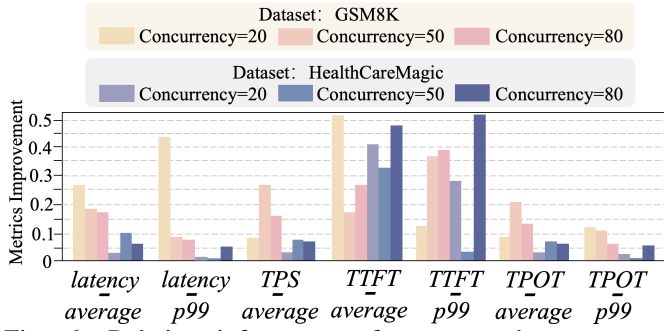


Fig. 6: Relative inference performance enhancement of Llama3-8B using LLMConf under different concurrency levels (higher is better). We conduct experiments on both *GSM8K* and *HealthCareMagic* datasets.

of LLMs under different concurrency levels. For this section of the experiment, we deploy Llama3-8B on the LLM platform and set three levels of concurrency: 20, 50 and 80. We then test whether LLMConf still effectively improves the inference performance of LLMs under different concurrency scenarios across two datasets. Fig.6 demonstrates the improvement of Llama3-8B inference performance under different concurrency levels. We have two observations: (i) LLMConf can effectively improve the inference performance of LLMs across different concurrency scenarios. It enhances all performance metrics of Llama3-8B at all three concurrency levels on both datasets. (ii) On the *GSM8K* dataset, LLMConf achieves average performance metric improvements of 25.7%, 20.1% and 17.8% at the three concurrency levels, respectively. On the *HealthCareMagic* dataset, it improves by 11.7%, 8.8% and 19.2% at the three concurrency levels, respectively.

We also deploy Qwen2.5-14B on our experimental platform to test whether LLMConf still effectively enhances the inference performance of the LLM. Fig.7 shows that LLMConf improves the inference performance of Qwen2.5-14B across different datasets. On average, LLMConf achieves improvements of 22.2% and 27.8% on all performance metrics for *GSM8K* and *HealthCareMagic*, respectively. This indicates that LLMConf can enhance the inference performance of the LLM with different parameters and architectures.

The experimental results demonstrate that LLMConf exhibits excellent transferability across different concurrency levels and LLMs. Regarding different system conditions, we construct the objective function of each performance metric for tuning parameters. We only need to optimize every objective function to obtain the optimal parameter configuration. In summary, LLMConf is capable of rapidly recommending optimal configuration parameters across different concurrency levels and LLMs.

3) **Ablation Study** : In this subsection, we validate the effectiveness of each design in LLMConf.

Contribution of Knowledge-based Parameter Selection. vLLM exposes nearly a hundred configuration parameters, making it impossible to tune all of them, which may lead to the omission of configuration parameters that have a significant

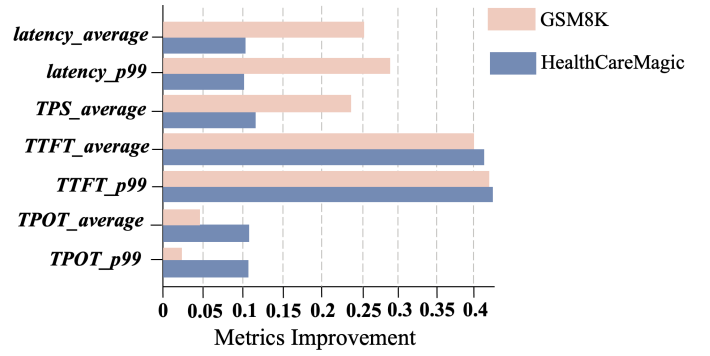


Fig. 7: Relative inference performance enhancement of Qwen2.5-14B using LLMConf across different datasets. Here, we set the concurrency level to 50.

impact on LLM inference performance. To identify tuning parameters and narrow down the search space, LLMConf introduces the Knowledge-based Parameter Selection Module.

To demonstrate the accuracy of the knowledge-enhanced method, we randomly select 40 configuration parameters for experimentation. Each parameter is assigned a random value within a reasonable range and we record both parameter values and performance metrics during each benchmarking. We collect 1000 data per experiment. Based on the collected data, we train a multi-output Random Forest regression model with 100 trees to predict the seven performance metrics. The importance of each configuration parameter is calculated by accumulating each parameter's MSE reduction weighted by the number of samples in split nodes across all decision tree node splits during training, with normalization ensuring the total importance sums to 1. Higher importance values indicate parameters that more substantially affect the joint prediction accuracy across all metrics.

Through multiple experiments with varying LLMs, concurrency levels and datasets, the results consistently align with Fig.8, showing that parameters selected by our knowledge-enhanced method rank among the top 11 in importance. Moreover, the six most critical parameters remain consistent across experiments, yet SCOOT [13] misses *gpu-memory-utilization* and *swap-space*. The above experiments confirm the accuracy and comprehensiveness of our parameter selection without missing potentially effective configurations.

To demonstrate the effectiveness of the knowledge-enhanced method, we compare different configuration parameter selection methods on optimizing latency and throughput. We conduct data collection based on the tuning parameters and their ranges selected from SCOOT [13] and LLMConf, respectively. We then perform performance modeling on the collected data using the same modeling approach and use the multi-objective optimization algorithm proposed in our paper to recommend optimal configuration parameters.

In the experiment, we select Llama3-8B, set the concurrency level to 50 and choose the *GSM8K* dataset. Fig.9 indicates that the improvement in latency and throughput is more significant

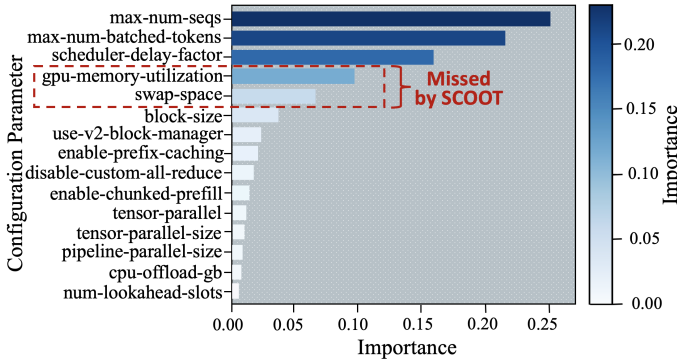


Fig. 8: Importance analysis of vLLM configuration parameters using Random Forest based on Llama3-8B (arrange in descending order of importance). Here, we set the concurrency level to 50 and choose the *GSM8K* dataset.

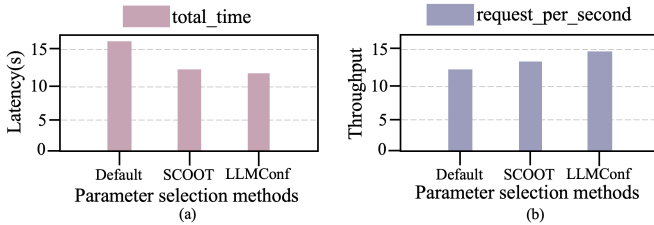


Fig. 9: (a) shows the comparison of *total_time* (the total time for all inference requests) under different parameter selection methods. (b) shows the comparison of *request_per_second* (the number of inference requests processed per second) under different parameter selection methods. Here, Default refers to the default configuration parameter values provided by vLLM.

under the knowledge-based parameter selection approach. The experimental results prove that the parameters selected by the knowledge-based parameter selection approach are beneficial for improving LLM inference performance. Our approach also improves the efficiency of the optimal solution search (**Effective resolution of C2**).

Contribution of Performance Modeling. We need to construct the objective function of each performance metric for tuning parameters. To more accurately predict performance metrics based on configuration parameters, LLMConf introduces an automated machine learning tool called TPOT [10]. To demonstrate its effectiveness, we compare it with other machine learning models. Table III presents a comparison of the differences in MSE when using different models for modeling. We model every performance metric, but due to page limitations, we show only one performance metric from each category. Among all the models, the TPOT model achieves the highest accuracy with an average MSE of 10.6 across 7 performance metrics. Therefore, we can accurately predict the impact of configuration parameter changes on LLM inference performance, effectively solving the cold start problem (**Effective resolution of C1**).

Contribution of Multi-objective Optimization. When the number of optimization objectives increases, existing multi-

objective optimization algorithms struggle to find high-quality solutions due to the expanded search space and are prone to getting stuck in local optima. To simultaneously optimize the predictive functions corresponding to each performance metric, LLMConf introduces the multi-objective optimization module. To demonstrate its effectiveness, we conduct experiments using Llama3-8B deployed on our platform, with a concurrency setting of 50 and the *GSM8K* dataset. Fig. 4 shows the comparative results of different optimization algorithms. The experimental results show that LLMConf optimizes multiple performance metrics simultaneously, facilitating a comprehensive improvement in the inference performance of the LLM (**Effective resolution of C3**).

TABLE III: MSE Comparison Across Models.

Models	<i>latency_average</i>	<i>TPS_average</i>	<i>TTFT_average</i>	<i>TPOT_p99</i>
CNN	3.61	1.33	0.07	0.05
Random Forest	3.08	1.37	0.06	0.04
XGBoost	3.26	1.56	0.07	0.05
SVM	2.88	2.36	0.07	0.04
MLP	3.49	1.35	0.06	0.04
TPOT	2.86	1.22	0.06	0.03

VI. RELATED WORK

A. Advanced LLM Inference Engines

Existing inference engines such as vLLM [3] and TensorRT-LLM [14] integrate a series of advanced techniques to enhance the inference performance of LLMs. For example, these inference engines utilize the paged attention [16] to break long sequences into smaller chunks, thereby reducing memory usage. Additionally, the engines implement chunked prefill [28] to process input data in parallel, effectively minimizing wait times. However, these advanced techniques involve numerous tuneable parameters, the default values typically only support standard operating conditions. To boost their performance, further optimization is often required.

B. Configuration Tuning Techniques

Automatic configuration tuning can optimize performance of applications and improve resource utilization, enhancing system performance and stability. Configuration tuning has been widely applied across various fields, including database tuning [29], big data analysis application configuration tuning [6, 30] and computer system resource allocation[31].

However, research focusing on the configuration parameter tuning of LLM inference engines is still relatively scarce. SCOOT [13], as a typical representative, utilizes BO and Random Forest to tune parameters of vLLM. Nevertheless, it lacks a comprehensive investigation into performance-critical parameter selection and configuration scopes, resulting in sub-optimal results compared with LLMConf.

VII. CONCLUSION

In this study, we propose LLMConf, a configuration parameter optimization method for improving LLM inference service

quality. We demonstrate that optimizing the configuration parameters of the inference engine plays a significant role in enhancing LLM inference performance. The innovation of LLMConf lies in its knowledge-enhanced approach to selecting the configuration parameters to be tuned and reducing the search space of configuration parameter combinations. Additionally, it utilizes the multi-objective optimization module based on the NSGA-III algorithm to optimize 7 important performance metrics. Experiments validate the effectiveness and transferability of LLMConf.

VIII. ACKNOWLEDGMENTS

The research is supported by National Key Research and Development Program of China under Grant 2024YFB4505904, the Guangdong Basic and Applied Basic Research Foundation (No.2023B1515020054), and the National Natural Science Foundation of China (No.62272495). Pengfei Chen and Chuanfu Zhang are the corresponding authors.

REFERENCES

- [1] T. B. Brown, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.
- [2] A. A. Chien, L. Lin *et al.*, "Reducing the carbon impact of generative ai inference (today and in 2035)," in *HotCarbon*, 2023, pp. 1–7.
- [3] "vllm", 2024. [Online]. Available: <https://github.com/vllm-project/vllm>
- [4] Y. He, M. Xu *et al.*, "Uellm: A unified and efficient approach for llm inference serving," in *ICSOC*, 2024.
- [5] A. K. Kakolyris, D. Masouros *et al.*, "Slo-aware gpu frequency scaling for energy efficient llm inference serving," *arXiv preprint arXiv:2408.05235*, 2024.
- [6] H. Dou, Y. Wang, Y. Zhang, and P. Chen, "Deepcat: A cost-efficient online configuration auto-tuning approach for big data frameworks," in *ICPP*, 2022.
- [7] M. Rahmani, R. Romero *et al.*, "Strategies to reduce the number of variables and the combinatorial search space of the multistage transmission expansion planning problem," *IEEE TPWRS*, pp. 2164–2173, 2012.
- [8] N. Gunantara, "A review of multi-objective optimization: Methods and its applications," *COGENT ENG*, 2018.
- [9] R. Hendel, M. Geva *et al.*, "In-context learning creates task vectors," *arXiv preprint arXiv:2310.15916*, 2023.
- [10] R. S. Olson and J. H. Moore, "Tpot: A tree-based pipeline optimization tool for automating machine learning," in *AutoML*. PMLR, 2016.
- [11] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based non-dominated sorting approach, part i: solving problems with box constraints," *IEEE TEVC*, pp. 577–601, 2013.
- [12] H. Jain and K. Deb, "An evolutionary many-objective optimization algorithm using reference-point based non-dominated sorting approach, part ii: Handling constraints and extending to an adaptive approach," *IEEE TEVC*, vol. 18, no. 4, pp. 602–622, 2013.
- [13] K. Cheng, Z. Wang *et al.*, "Scoot: Slo-oriented performance tuning for llm inference engines," in *Proceedings of the ACM on Web Conference*, 2025.
- [14] "Tensorrt-llm", 2024. [Online]. Available: <https://github.com/NVIDIA/TensorRT-LLM>
- [15] Z. Lu, Z. Yin, M. He *et al.*, "B-vllm: A vision large language model with balanced spatio-temporal tokens," *arXiv preprint arXiv:2412.09919*, 2024.
- [16] W. Kwon, Z. Li, and S. Zhuang, "Efficient memory management for large language model serving with page-dattention," in *SOSP*, 2023.
- [17] G. Singh and A. K. Chaturvedi, "Hybrid modified particle swarm optimization with genetic algorithm (ga) based workflow scheduling in cloud-fog environment for multi-objective optimization," *Cluster Computing*, 2024.
- [18] Z. Wang, S. Li *et al.*, "Revisiting slo and goodput metrics in llm serving," *arXiv preprint arXiv:2410.14257*, 2024.
- [19] B. Cai, Y. Liu *et al.*, "Hunter: an online cloud database hybrid tuning system for personalized requirements," in *SIGMOD*, 2022.
- [20] N. Siegmund, S. S. Kolesnikov *et al.*, "Predicting performance via automated feature-interaction detection," in *ICSE*. IEEE, 2012, pp. 167–177.
- [21] P. Jamshidi, N. Siegmund *et al.*, "Transfer learning for performance modeling of configurable systems: An exploratory analysis," in *ASE*. IEEE, 2017, pp. 497–508.
- [22] Meta-Llama-3, <https://github.com/meta-llama/llama3>, 2024.
- [23] Q. Team, "Qwen2.5: A party of foundation models," September 2024. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5/>
- [24] B. Zhang, X. Zhang *et al.*, "Cot-based synthesizer: Enhancing llm performance through answer synthesis," *arXiv preprint arXiv:2501.01668*, 2025.
- [25] A. Nishat and A. Mustafa, "A novel approach to emotion classification with llama3-8b: Integrating lora for efficient training," *Aitoz Multidisciplinary Review*, 2024.
- [26] Q.-W. Zhang, H. Wang *et al.*, "Cjeval: A benchmark for assessing large language models using chinese junior high school exam data," *arXiv preprint arXiv:2409.16202*, 2024.
- [27] J. D. M.-W. C. Kenton and L. K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of naacL-HLT*, 2019.
- [28] A. Agrawal, N. Kedia *et al.*, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," *arXiv preprint arXiv:2403.02310*, 2024.
- [29] J. Zhang, Y. Liu *et al.*, "An end-to-end automatic cloud database tuning system using deep reinforcement learning," in *SIGMOD*, 2019.
- [30] A. Fekry, L. Carata *et al.*, "To tune or not to tune? in search of optimal configurations for data analytics," in *SIGKDD*, 2020, pp. 2494–2504.
- [31] N. Akhtar, A. Raza, V. Ishakian, and I. Matta, "Cose: Configuring serverless functions using statistical learning," in *IEEE INFOCOM*, 2020.